

Enterprise On-Prem LLM Platform

Next Features Plan

Technical Planning Document

Status: Planning only — no code changes made

Scope: Four cross-cutting enhancements to be layered on top of the existing POC

Principle: Each feature is assigned to the layer(s) it naturally belongs in; the existing request flow and IMF contract are preserved

Document created: 2026-08-03 | Revised: 2026-08-04 — removed separate user_store service; user/key/role DB access consolidated into Admin Portal API (:8084).

Next step: pick a starting feature (recommended: Phase 1 — DB + Admin Portal API extensions) and create a spec.

Table of Contents

1. Feature Overview & Layer Mapping

#	Feature	Primary Layer(s)	Secondary Layers
1	RBAC	Security Layer (enforcement)	API Gateway (identity resolution), Admin Portal (management), DB
2	Chat UI	Portal UI (new Chat view)	Admin Portal API (new /portal/chat/* endpoints)
3	Persistent DB	Admin Portal API (owns DB access; exposes user/key/role endpoints)	Audit Store (migrate SQLite → Postgres), all services that read users/keys
4	Per-User API Keys + Model Entitlements	API Gateway (key resolution)	Security Layer (enforcement), Admin Portal (admin CRUD), DB

The four features are interdependent: **DB must be built first** because RBAC and API key management both need it. Chat UI is independent and can be built in parallel once the Admin Portal has the right endpoints.

2. Feature 1 — RBAC (Role-Based Access Control)

2.1 Current State

The Security Layer already has a stub policy check (Stage 4) that validates user.roles against a hardcoded set: {developer, analyst, admin}. However:

- Roles are passed in from the request itself (no server-side lookup).
- There is no user store — identity is purely the API key.
- There is no concept of "what can this role do" beyond a binary allow/deny for the whole API.
- The Admin Portal has no user management UI.

2.2 Target State

A proper RBAC model with:

- Users stored in the database, each with one or more roles.
- Roles define permissions (which models, which endpoints, what rate limits).
- Enforcement happens server-side at the API Gateway (identity resolution) and Security Layer (policy enforcement), not based on client-supplied claims.

2.3 Role Definitions

Role	Description	Model Access	Admin Access
viewer	Read-only; can query audit logs	None	No
analyst	Can chat; limited to chat + summarization tasks	Permitted models only	No
developer	Can chat; all task types; can use all models they are entitled to	Permitted models only	No
admin	Full access; manages users, keys, roles, model entitlements	All models	Yes

2.4 Changes by Layer

API Gateway (:8080)

- Identity resolution middleware — on every authenticated request, call `GET /portal/keys/resolve?key={X-API-Key}` on the Admin Portal API to resolve user_id, roles, and model_entitlements.
- Populate the IMF user block (user_id, roles, department) from the resolved profile instead of from the request payload.
- Return HTTP 401 if the key is not found or revoked.
- Return HTTP 403 if the key is found but the user has no active roles.
- Cache the key-to-user mapping in memory (short TTL, e.g. 30 seconds) to avoid a round-trip on every request.

Security Layer (:8081)

- Promote Stage 4 (Policy Check) from stub to real enforcement.
- Read user.roles from the IMF (now populated server-side by the Gateway).
- Load a policy matrix (YAML or DB table) that maps (role, task_type) → allow/deny.

Example policy matrix:

Role	chat	code	reasoning	summarization	translation	Admin endpoints
viewer	X	X	X	X	X	X
analyst	✓	X	X	✓	✓	X
developer	✓	✓	✓	✓	✓	X
admin	✓	✓	✓	✓	✓	✓

- Returns HTTP 403 policy_denied with reason: insufficient_role_for_task.
- The policy matrix file path is configurable via the POLICY_MATRIX_PATH environment variable.

Admin Portal API (:8084) — new RBAC management endpoints

POST	/portal/users/	# create user
GET	/portal/users/	# list users
GET	/portal/users/{user_id}	# get user detail
PATCH	/portal/users/{user_id}/roles	# assign / remove roles
DELETE	/portal/users/{user_id}	# deactivate user
GET	/portal/roles/permissions	# list available roles + their permissions

- All management endpoints require the admin role (enforced via session or admin API key).

Portal UI (:5173) — new Admin views

- Users tab — list users, create user, assign roles, deactivate.
- Roles tab — view role-to-permission matrix (read-only in POC).

3. Feature 2 — Interactive Chat UI

3.1 Current State

- The Admin Portal Playground (POST /portal/playground/chat) already proxies chat to the API Gateway.
- The Portal UI has some components but no dedicated interactive chat view with model selection, session history, or streaming support.

3.2 Target State

A dedicated Chat view in the Portal UI where:

- The user selects a model from a dropdown (populated from the Model Registry).
- Messages are displayed in a chat-bubble layout with user / assistant distinction.
- The session history is kept in-browser for the current tab (not persisted to the DB in POC scope).
- Streaming responses are displayed token-by-token as they arrive.
- The user's role-permitted models are shown (unapproved models are greyed out or hidden).

3.3 Changes by Layer

Portal UI (:5173) — new ChatView component

- /chat route — new page accessible from the sidebar.
- Model selector — dropdown populated by GET /portal/models; filters to models the current user is entitled to.
- Chat window — scrollable message list with user bubbles (right-aligned) and assistant bubbles (left-aligned).
- Input bar — textarea + send button; Enter key submits; Shift+Enter inserts a newline.
- Streaming — uses EventSource or fetch with ReadableStream to consume SSE from the backend; renders tokens as they arrive.
- Session controls — a "New chat" button clears in-browser history.
- Error states — surfaces 400/403/429/502 errors inline in the chat (e.g. "Access denied for this model").

Admin Portal API (:8084) — new Chat proxy endpoints

POST /portal/chat/completions (non-streaming)	# proxy to API Gateway /v1/chat/completions
POST /portal/chat/completions/stream	# proxy streaming SSE to API Gateway
GET /portal/chat/models	# filtered model list for current user's entitlements

- POST /portal/chat/completions — accepts { model, messages, temperature? }, attaches the user's resolved API key, and forwards it to the API Gateway.
- POST /portal/chat/completions/stream — streams SSE chunks back to the UI using StreamingResponse.
- GET /portal/chat/models — calls the Model Registry and filters by the user's model_entitlements.

API Gateway (:8080)

- No new endpoints needed — the existing /v1/chat/completions handles both streaming and non-streaming.
- The Admin Portal's new chat endpoints authenticate on behalf of the user using the user's API key (resolved from session or passed in the request header).

4. Feature 3 — Persistent Database

4.1 Current State

Service	Storage	Limitation
Audit Store	SQLite (audit.db)	Not concurrent-safe at scale; single file
Model Registry	JSON file (models.json)	No queries; no relations
Security Layer	In-memory (patterns loaded at startup)	Stateless — no user data
All services	No user/key/role storage at all	Blocks RBAC + API key features

4.2 Target State

A single PostgreSQL instance (or SQLite in strict local mode for POC) acts as the shared relational store. Rather than a separate user_store microservice, the Admin Portal API (:8084) is extended to own all user/key/role DB access and expose the necessary internal resolution endpoints. This keeps the service count down and avoids adding a new port.

The Admin Portal API therefore has two distinct responsibilities:

1. Admin-facing management — create/update users, assign roles, manage API keys (existing intent).
2. Internal resolution — a lightweight /portal/keys/resolve endpoint used by the API Gateway on every request to look up a key's owner and entitlements.

4.3 Admin Portal API — Extended as the DB-Backed Service

Responsibility: own all user, role, and API key data. Connects directly to PostgreSQL. Exposes endpoints consumed both by the Portal UI (admin management) and by the API Gateway (key resolution).

New endpoints added to Admin Portal API (:8084):

```
# Internal key resolution (called by API Gateway on every request)
GET    /portal/keys/resolve?key={api_key}      # resolve key → user profile +
entitlements

# User management (admin UI)
POST   /portal/users/                          # create user
GET    /portal/users/                          # list users
GET    /portal/users/{user_id}                 # get user detail
PATCH /portal/users/{user_id}                 # update user (roles, status)
DELETE /portal/users/{user_id}                 # deactivate user (soft-delete)

# Role management (admin UI, read-only in POC)
GET    /portal/roles/                          # list available roles + their
permissions
GET    /portal/roles/{role}/permissions        # get permissions for a specific
role

# API Key management (admin UI)
POST   /portal/users/{user_id}/keys           # generate new API key for user
```



```
GET    /portal/users/{user_id}/keys          # list keys for user
DELETE /portal/users/{user_id}/keys/{key_id} # revoke a key
PATCH /portal/users/{user_id}/keys/{key_id}/models # update model entitlements
for key
```

Auth on new endpoints:

- /portal/keys/resolve — protected by the existing ADMIN_PORTAL_INTERNAL_KEY (service-to-service, never exposed externally).
- All /portal/users/* and /portal/roles/* endpoints — require the admin role (enforced via session or admin API key header).

Storage: Admin Portal API connects to PostgreSQL via SQLAlchemy. The DATABASE_URL env var is new for this service.

4.4 Database Schema

The full DDL is provided in Section 10 — Database Schema.

4.5 Audit Store Migration

- Current: SQLite.
- Target: keep SQLite for local POC (zero friction); add a DB_BACKEND=postgres environment variable that switches the Audit Store to use PostgreSQL via SQLAlchemy.
- No schema changes to existing audit tables.
- A migration script (scripts/migrate-audit-db.py) moves existing audit.db records to Postgres if needed.

4.6 Model Registry Migration

- Current: flat JSON file.
- Target: PostgreSQL table (dedicated table in the same DB owned by the Admin Portal API).
- The Model Registry service switches its storage backend from JSON file to DB when STORAGE_BACKEND=postgres is set.
- Existing models.json seed data can be imported by a one-time migration script.

4.7 Docker Compose Addition

docker-compose.local.yml addition:

```
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: llm_platform
    POSTGRES_USER: llm_user
    POSTGRES_PASSWORD: llm_pass
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
```

```
- ./seed/init.sql:/docker-entrypoint-initdb.d/init.sql
```

5. Feature 4 — Per-User API Keys with Model Entitlements

5.1 Current State

- One global API key (poc-secret-key) shared by all callers.
- No concept of "which user is calling" beyond what the caller claims in the IMF user block.
- No model-level access control — if you have the key, you can use any model.

5.2 Target State

- Each user in the platform DB (owned by Admin Portal API) can have one or more API keys.
 - A status (active, revoked, expired).
 - An optional expiry date.
 - A set of model entitlements — which models (by name, e.g. llama3.2:3b) this key is permitted to use.
 - A rate limit override (optional — if absent, uses the role's default).
- Admins manage keys via the Admin Portal UI.
- The API Gateway resolves the inbound X-API-Key against the Admin Portal API (/portal/keys/resolve) to get the full user profile.
- The Intelligent Router checks routing.model_entitlements (populated by the Gateway from the DB) before dispatching.

5.3 Key Lifecycle

```
Admin creates user → Admin generates API key for user → Admin assigns model
entitlements
→ User calls API with X-API-Key → Gateway resolves key → IMF populated with
entitlements
→ Router validates selected model is in entitlements → Allow / 403
```

5.4 Changes by Layer

API Gateway (:8080)

- On every request, call GET /portal/keys/resolve?key={X-API-Key} on the Admin Portal API.
- Populate the IMF user block: user_id, roles, department, model_entitlements.
- Add model_entitlements as a new field on the IMF user block.
- Reject with HTTP 401 if the key is not found or is revoked/expired.
- Cache resolved key profile (30-second TTL in-process) to reduce Admin Portal load.

Intelligent Router (:8082)

- After model selection (Stage 3), verify the selected model is in request.user.model_entitlements.
- If not entitled: return HTTP 403 model_not_entitled with {"allowed_models": [...]}.
 - If model_entitlements is empty or null (backward-compat): allow all models (preserves existing behaviour during rollout).

Admin Portal API (:8084) — new key management endpoints

These are the same endpoints already listed in Section 4.3 above. No duplication — they live in the Admin Portal API service.

POST	/portal/users/{user_id}/keys	# generate new API key
GET	/portal/users/{user_id}/keys	# list keys for user
DELETE	/portal/users/{user_id}/keys/{key_id}	# revoke key
PATCH	/portal/users/{user_id}/keys/{key_id}/models	# update model entitlements for key

Portal UI (:5173) — new Admin key management view

- Users → User Detail page — shows all API keys for the user, their status, expiry, and entitled models.
- Generate Key button — calls the Admin Portal to create a key; shows the raw key value once (copy-and-store warning).
- Revoke Key button — sets status to revoked immediately.
- Model Entitlements selector — multi-select checklist of available models; saved per key.

6. Cross-Feature Dependencies & Build Order

```

Phase 1 _____
[DB] Add PostgreSQL to docker-compose.local.yml
[DB] Extend Admin Portal API with DB connection (SQLAlchemy) + user/key/role
tables
[DB] Seed initial admin user + poc-secret-key (backward compat)

Phase 2 _____
[RBAC] API Gateway: key-to-user resolution via Admin Portal API
/portals/keys/resolve
[RBAC] Security Layer: promote policy check to role+task matrix
[KEYS] API Gateway: populate model_entitlements in IMF user block
[KEYS] Router: enforce model entitlements (Stage 3 extension)

Phase 3 _____
[ADMIN] Admin Portal API: RBAC management endpoints (/portal/users/*,
/portals/roles/*)
[ADMIN] Admin Portal API: key management endpoints (/portal/users/{id}/keys/*)
[ADMIN] Portal UI: Users tab, Roles tab, Key management view

Phase 4 _____
[CHAT] Admin Portal API: /portal/chat/* proxy endpoints
[CHAT] Portal UI: Chat view with model selector + streaming

Phase 5 (optional, if DB migration desired) _____
[DB] Audit Store: Postgres backend switch
[DB] Model Registry: Postgres backend switch
[DB] Migration scripts for existing SQLite / JSON data

```

Blocking Dependencies

- Phase 2 requires Phase 1 (Admin Portal DB layer must exist before the Gateway can resolve keys).
- Phase 3 requires Phase 2 (admin endpoints need RBAC to protect them).
- Phase 4 is independent of Phases 2–3 but needs Phase 1 for entitlement-filtered model lists.
- Phase 5 is fully optional and can be done at any time without breaking anything.

7. New Environment Variables

All new variables are to be added to local.env:

```
# --- Admin Portal API additions (port 8084 — no new port) ---
DATABASE_URL=postgresql://llm_user:llm_pass@localhost:5432/llm_platform
# fallback for POC without Postgres:
# DATABASE_URL=sqlite:///./admin_portal.db
ADMIN_PORTAL_INTERNAL_KEY=poc-portal-internal-key # used by API Gateway to
call /portal/keys/resolve

# --- API Gateway additions ---
ADMIN_PORTAL_URL=http://localhost:8084 # already exists for other portal calls;
reused
KEY_CACHE_TTL_SECONDS=30

# --- Security Layer additions ---
POLICY_MATRIX_PATH=policy_matrix.yaml

# --- Audit Store (optional migration) ---
# DB_BACKEND=sqlite (default, existing behaviour)
# DB_BACKEND=postgres
# DB_BACKEND_URL=postgresql://llm_user:llm_pass@localhost:5432/llm_platform

# --- Model Registry (optional migration) ---
# STORAGE_BACKEND=json (default, existing behaviour)
# STORAGE_BACKEND=postgres
```

8. New Service Ports & Infrastructure

New Component	Port	Notes
PostgreSQL	:5432	Added to docker-compose.local.yml

No new Python services are introduced. The Admin Portal API (:8084) is extended in-place to own DB access and the new endpoints. The startup order in run-local.ps1 is unchanged — PostgreSQL is a Docker Compose dependency, not a Python service.

9. IMF Extensions

The IMF user block gains two new optional fields (backward-compatible — if absent, existing behaviour is preserved):

```
"user": {  
  "user_id": "string",          // existing  
  "department": "string",       // existing  
  "roles": ["string"],          // existing – now populated server-side  
  "auth_method": "string",      // existing  
  
  // NEW  
  "key_id": "string",           // which specific key was used  
  "model_entitlements": ["llama3.2:3b", "..."], // models this key is allowed to  
use  
  "rate_limit_override": null   // optional int (requests/min); null = use role  
default  
}
```

No other IMF blocks change. All existing services that don't read these fields continue to work without modification.

10. Database Schema

users

```
CREATE TABLE users (
  user_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username     TEXT UNIQUE NOT NULL,
  email        TEXT UNIQUE,
  department   TEXT,
  status       TEXT NOT NULL DEFAULT 'active',  -- active | inactive
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

roles (static seed data; not user-managed in POC)

```
CREATE TABLE roles (
  role_name    TEXT PRIMARY KEY,  -- viewer | analyst | developer | admin
  description  TEXT
);
```

user_roles (many-to-many)

```
CREATE TABLE user_roles (
  user_id      UUID REFERENCES users(user_id) ON DELETE CASCADE,
  role_name    TEXT REFERENCES roles(role_name) ON DELETE CASCADE,
  assigned_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, role_name)
);
```

api_keys

```
CREATE TABLE api_keys (
  key_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  key_hash     TEXT UNIQUE NOT NULL,  -- bcrypt / SHA-256 of the raw key; raw
key never stored
  key_prefix   TEXT NOT NULL,         -- first 8 chars of raw key, for display
(e.g. "poc-secr")
  label       TEXT,                  -- human-readable name, e.g. "dev laptop
key"
  status       TEXT NOT NULL DEFAULT 'active',  -- active | revoked | expired
  expires_at   TIMESTAMPTZ,          -- null = never expires
  rate_limit_rpm INT,                -- null = use role default
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_used_at TIMESTAMPTZ
);
```

key_model_entitlements (which models a key can use)

```
CREATE TABLE key_model_entitlements (
  key_id       UUID REFERENCES api_keys(key_id) ON DELETE CASCADE,
  model_name    TEXT NOT NULL,        -- e.g. "llama3.2:3b"
  granted_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (key_id, model_name)
);
```

role_permissions (policy matrix — static seed; maps role+task → allow/deny)

```
CREATE TABLE role_permissions (
  role_name      TEXT REFERENCES roles(role_name) ON DELETE CASCADE,
  task_type      TEXT NOT NULL,          -- chat | code | reasoning |
summarization | translation
  allowed        BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (role_name, task_type)
);
```

10.1 Seed Data (seed/init.sql)

```
INSERT INTO roles VALUES
  ('viewer',    'Read-only access'),
  ('analyst',   'Chat and summarization'),
  ('developer', 'Full task access, no admin'),
  ('admin',     'Full access including user management');

-- role_permissions seed (matches policy matrix in Section 2.3)
INSERT INTO role_permissions VALUES
  ('analyst',   'chat',          true),
  ('analyst',   'summarization', true),
  ('analyst',   'translation',   true),
  ('developer', 'chat',          true),
  ('developer', 'code',          true),
  ('developer', 'reasoning',     true),
  ('developer', 'summarization', true),
  ('developer', 'translation',   true),
  ('admin',     'chat',          true),
  ('admin',     'code',          true),
  ('admin',     'reasoning',     true),
  ('admin',     'summarization', true),
  ('admin',     'translation',   true);

-- seed admin user and the existing poc-secret-key for backward compatibility
INSERT INTO users (user_id, username, email, department)
  VALUES ('00000000-0000-0000-0000-000000000001', 'admin', 'admin@local',
'platform');

INSERT INTO user_roles VALUES
  ('00000000-0000-0000-0000-000000000001', 'admin', NOW());

-- key_hash is SHA-256("poc-secret-key") — actual value generated by migration
script
INSERT INTO api_keys (key_id, user_id, key_hash, key_prefix, label, status)
  VALUES (
    '00000000-0000-0000-0000-000000000002',
    '00000000-0000-0000-0000-000000000001',
    '<sha256_of_poc-secret-key>',
    'poc-secr',
    'Legacy POC key',
    'active'
  );
```

```
-- entitle the seed key to llama3.2:3b
INSERT INTO key_model_entitlements VALUES
('00000000-0000-0000-0000-000000000002', 'llama3.2:3b', NOW());
```

11. What Stays Unchanged

The following are not touched by any of the four features described in this document:

Component	Reason Unchanged
Intelligent Router — Stages 1–5	Only Stage 3 gets a model entitlement check appended
Inference Adapter	No auth, no user context — purely a thin Ollama wrapper
Cache Service	No user-specific caching (shared cache, keyed by content + model + task)
Audit Store schema	Existing tables and query API unchanged; DB backend is opt-in
Model Registry endpoints	Unchanged; optional DB backend is opt-in
Agent Framework	Stub remains a stub; not in scope
IMF request, governance, routing, cache, response blocks	No changes
model_matrix.yaml / task_classifier_rules.yaml / injection_patterns.yaml	No changes
Prometheus metrics naming	Existing metric names unchanged; new services add their own
Streaming path	Works as-is; Chat UI adds a new consumer, no backend change
K8s / Helm charts	Phase 2 concern; all local-first
POC constraints list	OIDC / Vault / OPA / etc. still deferred

Document created: 2026-08-03 | Revised: 2026-08-04 — removed separate user_store service; user/key/role DB access consolidated into Admin Portal API (:8084).

Next step: pick a starting feature (recommended: Phase 1 — DB + Admin Portal API extensions) and create a spec.